

백엔드 API 개발

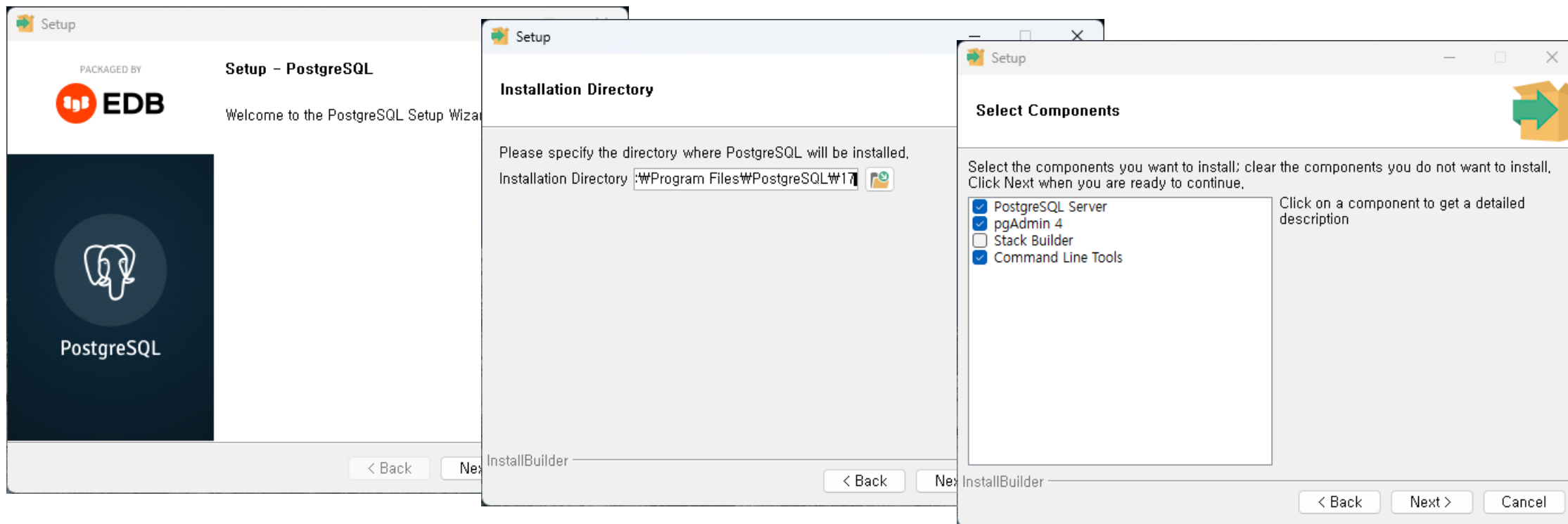


1. Database 준비

❖ 사용할 DB : Postgresql DB

- 다운로드 : <https://www.enterprisedb.com/downloads/postgres-postgresql-downloads>
- 각 운영체제에 맞는 것을 다운로드받아서 설치
- 이 교안에서는 윈도우 기준으로 설명

❖ 설치 과정



1. Database 준비

❖ 설치 과정 (이어서)

Setup

Data Directory

Please select a directory under which to store your data.

Data Directory 

Setup

Password

Please provide a password for the database superuser (postgres).

Password

Retype password

Setup

Port

Please select the port number the server should listen on.

Port

InstallBuilder

< Back Next > Cancel

Setup

Advanced Options

Select the locale to be used by the new database cluster.

Locale

InstallBuilder

< Back Next > Cancel

1. Database 준비

❖ .env 파일에 다음과 같이 연결 문자열 설정

```
POSTGRES_CONNECTION_STRING=postgresql://postgres:패스워드@localhost:5432/postgres
```

❖ postgresql-mcp 설정

- .mcp.json 파일에 다음과 같이 mcp 설정

```
{  
  "mcpServers": {  
    .....(생략)  
    "postgresql-mcp": {  
      "command": "npx",  
      "args": ["-y", "@henkey/postgres-mcp-server"]  
    }  
  }  
}
```

- 설정 후 claude를 재시작함

1. Database 준비

❖데이터베이스 연결 기능 테스트

- /mcp 명령어로 연결된 MCP 서버들의 목록 확인
- .env의 설정을 이용해 postgresql-mcp를 이용해 로컬 데이터베이스에 연결되는지 테스트해줘

● 테스트 결과

✓ postgresql-mcp 연결 성공

- PostgreSQL 17.6 (Windows 64-bit)에 성공적으로 연결
- 데이터베이스: postgres
- 사용자: postgres
- 서버 주소: ::1 (IPv6 localhost)
- 포트: 5432

.env 파일의 연결 문자열 `postgresql://postgres:asdf!2345@localhost:5432/postgres`을 사용하여 로컬 PostgreSQL 데이터베이스에 정상적으로 연결되었습니다. postgresql-mcp가 완전히 작동하는 것을 확인했습니다.

2. 데이터베이스 구성

❖미리 작성해둔 실행 계획대로 구성 진행

- @docs\W7-execution-plan.md 의 1단계의 1.1 환경 설정 및 초기화의 첫번째 단계인 Postgresql 설치 및 기본 설정을 진행해줘. 완료되었다면 검수리스트 체크박스에 체크해야 해.

1.1 환경 설정 및 초기화

PostgreSQL 설치 및 기본 설정

- [x] ****PostgreSQL 14+ 설치 확인 및 버전 검증****
 - ****완료 조건****: `psql --version`으로 14.x 이상 확인
 - ****의존성****: 없음
 - ****예상 시간****: 30분
 - ****✅ 완료****: PostgreSQL 17.6 설치 확인됨
- [x] ****PostgreSQL 서비스 시작 및 자동 시작 설정****
 - ****완료 조건****: 서비스 상태 active(running) 확인
 - ****의존성****: PostgreSQL 설치 완료
 - ****예상 시간****: 15분
 - ****✅ 완료****: 서비스 RUNNING 상태, AUTO_START 설정 확인됨
- [x] ****기본 설정 파일 백업 생성****
 - ****완료 조건****: `postgresql.conf`, `pg\_hba.conf` 백업 파일 생성
 - ****의존성****: PostgreSQL 설치 완료
 - ****예상 시간****: 10분
 - ****✅ 완료****: 백업 파일들이 성공적으로 생성됨

2. 데이터베이스 구성

❖ 데이터베이스 및 사용자 생성

- 다음으로 '데이터베이스 및 사용자 생성' 단계를 진행해줘.

데이터베이스 및 사용자 생성

- [x] **`team\_caltalk` 데이터베이스 생성**
 - **완료 조건**: ``\l`` 명령으로 데이터베이스 존재 확인
 - **의존성**: PostgreSQL 서비스 실행
 - **예상 시간**: 15분
 - **✅ 완료**: ``team\_caltalk`` 데이터베이스 생성 완료
- [x] **전용 데이터베이스 사용자 생성 및 권한 설정**
 - **완료 조건**: 사용자로 데이터베이스 접속 및 CRUD 작업 가능
 - **의존성**: 데이터베이스 생성 완료
 - **예상 시간**: 20분
 - **✅ 완료**: ``team\_caltalk\_user`` 생성 및 CRUD 권한 확인됨
- [x] **`btree\_gist` 확장 모듈 설치 및 활성화**
 - **완료 조건**: ``\dx`` 명령으로 확장 모듈 설치 확인
 - **의존성**: 데이터베이스 생성 완료
 - **예상 시간**: 10분
 - **✅ 완료**: btree_gist 버전 1.7 설치 완료

2. 데이터베이스 구성

❖스키마 적용 및 검증

- 1.2 스키마 적용 및 검증 단계를 진행해줘.

1.2 스키마 적용 및 검증

- [x] ****기존 스키마 파일 문법 검증****
 - ****완료 조건****: ``psql -f schema.sql --dry-run`` 성공
 - ****의존성****: 확장 모듈 설치 완료
 - ****예상 시간****: 15분
 - ****✅ 완료****: 스키마 파일 문법 검증 및 확장 모듈 확인 완료
- [x] ****테이블 생성 스크립트 실행****
 - ****완료 조건****: 6개 핵심 테이블 모두 생성 확인
 - ****의존성****: 스키마 문법 검증 완료
 - ****예상 시간****: 30분
 - ****✅ 완료****: 6개 테이블 (users, teams, team_members, schedules, schedule_participants, messages) 생성 완료
- [x] ****성능 최적화 인덱스 생성 실행****
 - ****완료 조건****: 22개 인덱스 모두 생성 확인 (``\di`` 명령)
 - ****의존성****: 테이블 생성 완료
 - ****예상 시간****: 20분
 - ****✅ 완료****: 21개 인덱스 생성 완료 (GIST, 복합 인덱스 포함)
- [x] ****트리거 및 유틸리티 함수 생성****
 - ****완료 조건****: ``update_updated_at_column()``, ``check_schedule_conflict()``, ``generate_invite_code()`` 함수 생성
 - ****의존성****: 테이블 생성 완료
 - ****예상 시간****: 30분
 - ****✅ 완료****: 3개 함수 및 3개 트리거, 2개 뷰 생성 완료

2. 데이터베이스 구성

❖ 성능 최적화 및 테스트 데이터 생성

- 1.3 성능 최적화 및 테스트 데이터 단계를 진행해줘.

1.3 성능 최적화 및 테스트 데이터

- [x] **PostgreSQL 성능 파라미터 조정**

- **완료 조건**: `shared\_buffers`, `work\_mem`, `max\_connections` 최적화
- **의존성**: 스키마 적용 완료
- **예상 시간**: 45분
- **✅ 완료**: shared_buffers=512MB, work_mem=8MB, max_connections=300, effective_cache_size=8GB, maintenance_work_mem=256MB 설정

- [x] **테스트 데이터 생성 (사용자 100명, 팀 50개)**

- **완료 조건**: 샘플 사용자, 팀, 일정, 메시지 데이터 생성
- **의존성**: 성능 설정 완료
- **예상 시간**: 1시간
- **✅ 완료**: 사용자 100명, 팀 50개, 팀원 431명, 일정 372개, 참가자 1600명, 메시지 5577개 생성

****Day 1 예상 총 소요시간****: 4.5시간

❖ 구성이 완료되었다면 Github Repo에 업데이트하자.

- 현재까지의 상태를 git commit 하고 리모트 리포지토리에 업데이트해줘.

3. Github Issue 생성

❖Github Issue란?

- GitHub 리포지토리 내에서 작업(task), 버그(bug), 기능 요청(feature request), 질문(question) 등을 추적하고 관리하기 위한 일종의 게시판 기능
- 소프트웨어 개발 프로젝트나 협업 과정에서 발생하는 이슈들을 정리하고 팀원들과 공유할 있게 함

❖Github issue의 주요 용도

- 버그 리포트: 발견한 오류나 문제점 보고
- 기능 제안: 새로운 기능이나 개선사항 제안
- 질문: 프로젝트 사용 중 궁금한 점 문의
- 작업 관리: 해야 할 일(To-do) 추적

❖주요 요소

- 라벨(Labels): 이슈를 분류 (예: bug, frontend, documentation)
- 담당자(Assignees): 특정 팀원에게 이슈 할당
- 마일스톤(Milestones): 이슈를 버전이나 목표별로 그룹화한 것
- 참조: 커밋, PR(Pull Request)과 연결 가능

3. Github Issue 생성

❖Github Issue 생성

전문화된 서브에이전트를 이용해 @docsW7-execution-plan.md 의 처리 단계를 유지하면서 @docsW7 디렉토리의 문서들을 참조하여 깃헙 이슈를 생성해줘.

반드시 지켜야할 것

- Title : Stage를 명시할 것
- Label : 종류, 영역, 복잡도
- Todo : 해야할 일 정보
- 완료 조건
- 기술적 고려사항
- 의존성
- 선행, 후행 작업

3. Github Issue 생성

❖생성된 Github Issue

is:issue state:open

Q

Labels

Milestones

New issue

☐ Open 8 Closed 0

Author ▾ Labels ▾ Projects ▾ Milestones ▾ Assignees ▾ ⌵ Newest ▾

☐ ☒ [Stage 9] 통합 테스트 및 성능 최적화 complexity-high integration performance testing

#16 · stepanowon opened 2 minutes ago

☐ ☒ [Stage 8] 실시간 채팅 UI 및 통합 인터페이스 구현 complexity-high frontend real-time-chat ui-components

#15 · stepanowon opened 3 minutes ago

☐ ☒ [Stage 7] 캘린더 뷰 및 일정 관리 UI 구현 complexity-high frontend schedule-management

#14 · stepanowon opened 4 minutes ago

☐ ☒ [Stage 6] 사용자 인증 및 팀 관리 UI 구현 authentication complexity-high frontend

#13 · stepanowon opened 5 minutes ago

☐ ☒ [Stage 5] 프론트엔드 기반 구조 및 상태 관리 설정 complexity-high frontend ui-components

#12 · stepanowon opened 5 minutes ago

☐ ☒ [Stage 4] 실시간 채팅 시스템 구현 backend complexity-high real-time-chat

#11 · stepanowon opened 6 minutes ago

☐ ☒ [Stage 3] 일정 관리 API 및 비즈니스 로직 구현 backend complexity-high schedule-management

#10 · stepanowon opened 6 minutes ago

☐ ☒ [Stage 2] 백엔드 기반 구조 및 인증 시스템 구현 authentication backend complexity-medium

#9 · stepanowon opened 7 minutes ago

4. 개발전 사전 작업

❖이슈를 처리할 때 사용할 Skill 생성

▪ .claude/skills/issue-resolver-backend/SKILL.md 예시

name: issue-resolver-backend

description: 깃헙 이슈 번호를 입력받아 백엔드 개발을 수행하는 사용자 정의 Skill

arguments: ISSUE_NUMBER

백엔드 Issue 해결 사용자 정의 Command

너는 Github Issue를 해결하는 유능한 백엔드 개발자이다. \${ISSUE_NUMBER}를 Argument로 전달받아 해당 번호의 Issue를 해결한다.

작업 내용

- 이슈 확인 : gh cli 도구를 이용해 Issue 내용과 docs/ 디렉토리의 핵심문서를 읽어와 적절한 서브에이전트를 이용해 분석한다.
- 자식 브랜치 분기 : feature-\${ISSUE_NUMBER} 형태의 자식 브랜치를 생성한다.
- 기존 코드 분석 : 적절한 서브에이전트를 사용해 swagger/swagger.json과 코드베이스(backend 디렉토리)의 코드를 분석한다.
- 이슈확인, 자식 브랜치 분기, 기존 코드 분석은 병렬로 실행한다.
- 계획 수립 : 기존 코드 분석한 결과와 분석된 Issue 내용을 바탕으로 독립적인 서브에이전트를 이용해 Issue 해결 계획을 수립한다.
- 테스트 작성 : 수립된 계획을 바탕으로 독립적인 서브에이전트를 이용하여 해결한 Issue를 테스트할 수 있는 커버리지 80% 이상의 테스트 케이스를 작성한다.
- 문제 해결 : 수립된 계획을 바탕으로 백엔드 개발에 적합한 서브에이전트를 선택해서 해결한다.
- 테스트 작성과 문제 해결은 병렬로 수행한다.
- 테스트 수행 : 독립적인 서브에이전트를 이용해 미리 작성한 테스트를 이용해 테스트한다.
- 테스트가 완결되면 feature-\${ISSUE_NUMBER} 브랜치를 리포지토리에 커밋, 푸시하고 PR을 생성한다.

4. 개발전 사전 작업

❖ 최상위 CLAUDE.md

- 리포지토리 수준에서 적용할 공통 지침
 - 예) 안드레카파시의 CLAUDE.md
 - 작업 수행 지침 : "오버엔지니어링 금지"
 - 커뮤니케이션 지침 : "모든 대화는 한국어로 할 것"
- 프로젝트 최상위 디렉토리 구조
- 작성한 문서들의 참조 정보 목록

❖ backend 디렉토리의 CLAUDE.md 지침

- 백엔드 개발을 진행하고 있는 하위 디렉토리(예: backend)에 CLAUDE.md 지침을 정의함
- 상위 CLAUDE.md에는 전체를 조망할 수 있는 지침을 정의
- 하위 CLAUDE.md에는 해당 디렉토리 영역에 적용할 지침을 정의
 - 백엔드 개발에 적용할 지침 예시
 - "배포할 때 변경할만한 것은 환경변수로 분리할 것"
 - "SOLID 원칙, Clean 아키텍처를 준수하도록 할 것"
 - 백엔드 아키텍처의 기본 골격

4. 개발전 사전 작업

❖백엔드 개발을 위한 mcp 설정 추가

- .mcp.json 예시

```
{
  "mcpServers": {
    "context7": {
      "command": "npx",
      "args": [ "-y", "@upstash/context7-mcp" ]
    },
    "postgresql-mcp": {
      "command": "npx",
      "args": [ "-y", "@henkey/postgres-mcp-server" ]
    }
  }
}
```

5. 백엔드 개발

❖ Skill 을 이용해 빠르게 개발

- 앞에서 만들어둔 issue-resolver-backend 스킴을 이용해 빠르게 개발
 - /issue-resolver-backend 이슈번호 추가적인 지침
- 테스트가 성공적으로 완료되면 feature 브랜치로 스테이징-커밋-푸시하고 PR(Pull Request) 생성
- Skill에 코드 리뷰까지 추가하여 main브랜치에 승인 후 병합까지 진행할 수 있음

❖ 디버깅

- AI가 모든 것을 해결해주려면 대단히 많은 토큰과 시간을 필요로 함
 - 원인을 파악하기 위해서 많은 토큰 소모, 반복적인 분석 작업을 하게 됨.
- 따라서 충실하게 로깅하도록 개발하고, 로그를 기반으로 분석하도록 함
 - 개발환경에서만 로깅, 프로덕션 환경에서는 로깅하지 않도록 logger 함수 작성할 것을 권장
- 가능하다면 단순한 에러메시지 뿐만 아니라 로그에 기록된 것을 직접 AI에게 알려주면 더 빠르게 디버깅할 수 있음

5. 백엔드 개발

❖ 완료된 백엔드 API 의 Swagger UI

- 총 35개 엔드포인트

Team CalTalk API

1.0.0 OAS 3.0

팀 중심의 일정 관리와 실시간 커뮤니케이션을 통합한 협업 플랫폼 API

[Contact Team CalTalk Support](#)

MIT

Servers

http://localhost:3000 - 로컬 개발 서버

Authorize

Filter by tag

사용자

사용자 정보 관리

^

GET

/api/users/{userId}

특정 사용자 정보 조회

🔒

✓

DELETE

/api/users/account

현재 사용자 계정 삭제

🔒

✓

GET

/api/users/profile

현재 사용자 프로필 조회

🔒

✓

PUT

/api/users/profile

현재 사용자 프로필 업데이트

🔒

✓

6. 백엔드 개발 완료후 수행할 것

❖백엔드 완료 후에 해야 할 것

- API 테스트 수행
- 프론트엔드 통합가이드 문서 작성
 - 백엔드의 사용 방법, CORS, 보안, Endpoint 등을 더욱 명확하게 설명하여 프론트엔드 개발시에 참조할 수 있도록 참조 문서 작성
 - 이 문서와 Swagger 정보를 활용해서 프론트엔드를 개발하게 될 것임
- Swagger, Swagger UI를 이용한 문서화 및 API Console 기능 생성
 - Swagger JSON, Swagger UI 생성
 - 백엔드 API에 구현된 Swagger의 Swagger JSON 문서의 내용으로 swagger/swagger.json 파일에 업데이트
- 문서화된 ERD, 스키마를 로컬 데이터베이스의 실제 스키마와 비교,검토
 - 백엔드 개발 시 AI가 DB 스키마를 변경할 수 있으므로 문서를 업데이트해주어야 함
- 보안 검토 수행
 - sast mcp와 같은 보안 스캔, 검토를 수행하는 mcp 사용할 수 있음
 - 다른 AI 도구(gemini-cli, codex 등)를 활용하여 보안 검토 수행하는 것을 적극 추천
 - OWASP Top 10(<https://owasp.org/www-project-top-ten/>)과 같은 공신력있는 문서를 참조하여 수행하도록 하면 더 좋은 효과를 기대할 수 있음

7. 정리

❖ DB 구성

- DB/사용자 생성 → 스키마 적용·검증 → 성능 튜닝·테스트 데이터 생성

❖ GitHub 이슈

- 실행 계획 기반으로 Stage가 드러나는 제목, 라벨/난이도, TODO, 완료조건, 기술 고려사항, 의존성/선후행 작업을 담아 생성

❖ 사전 작업

- 이슈를 해결할 수 있는 커스텀 커맨드 작성하고 커스텀 커맨드로 이슈 해결
 - feature-{번호} 브랜치 생성 → 코드/Swagger 병렬 분석 → 계획 수립 → 구현 → 테스트
 - 하위 CLAUDE.md로 백엔드 개발 전용 지침 지정 → SOLID 원칙, Clean 아키텍처 원칙 명시

❖ 개발 흐름

- 로그 중심 디버깅(개발환경 전용),
- 테스트 통과 시 스테이징→커밋·푸시 후 PR 생성

❖ 개발 완료 후

- Swagger 기반 API 테스트,
- 프론트엔드 개발을 위한 프론트엔드-백엔드 통합가이드(CORS/보안/엔드포인트) 작성
- Swagger 문서, ERD, 스키마 업데이트